# Creative Programming: Data Sources

Lecture 8

Narration

# Table of contents



Narration

## Random Dog Image

Click anywhere to load a new dog

Loaded

Narration

# (Re)visiting slides?

You can jump to any slide with the menu to the (bottom) left.

Did you miss a slide or want to revisit?
Open the narration tab while
studying to get an explanation of
difficult slides.

# Data in our world



Narration

# Data in our world


Narration

# Why data?

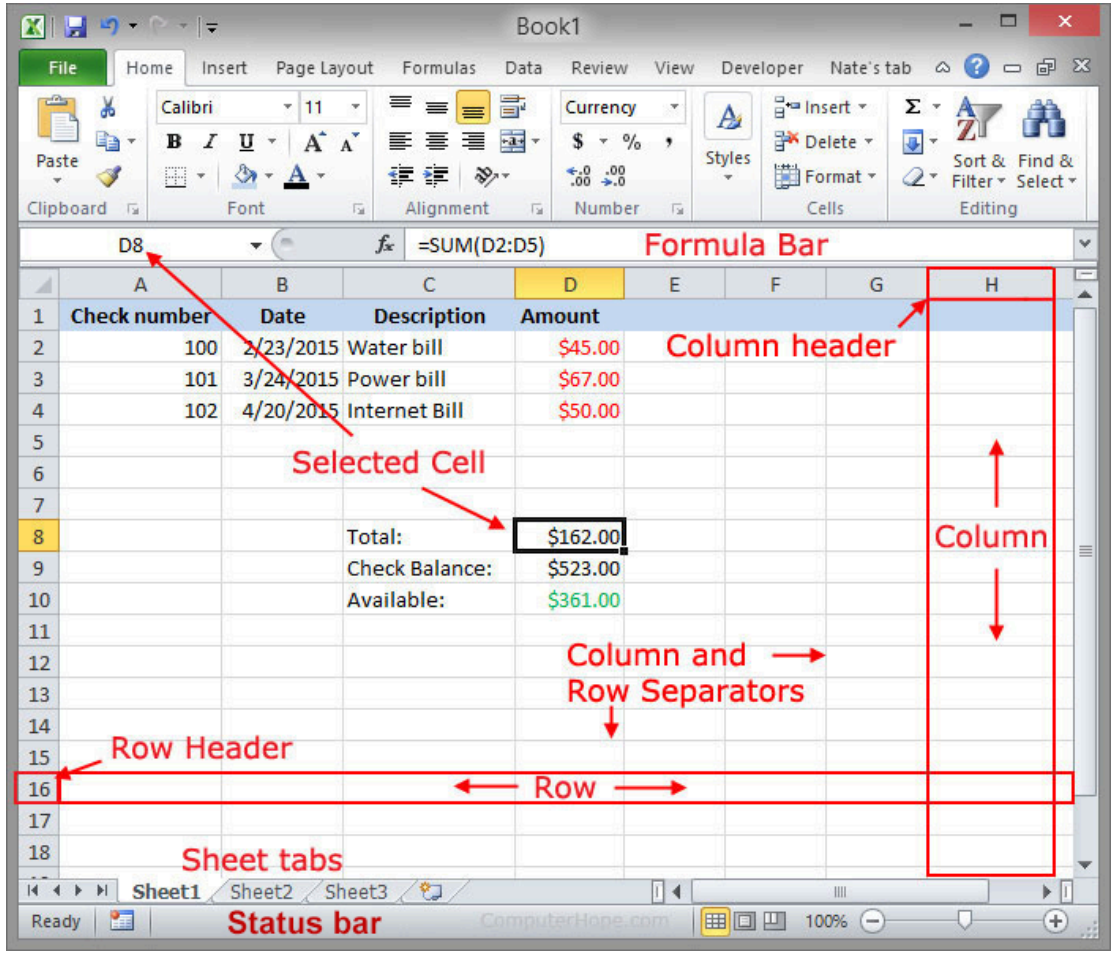
- Data helps us build projects that react to real information
- We can move from “hard-coded sketches” to “data-driven sketches”
- This is useful for art, games, visual journalism, installations, and project work
- NOTE: using data sources is not required for the final project!

Narration

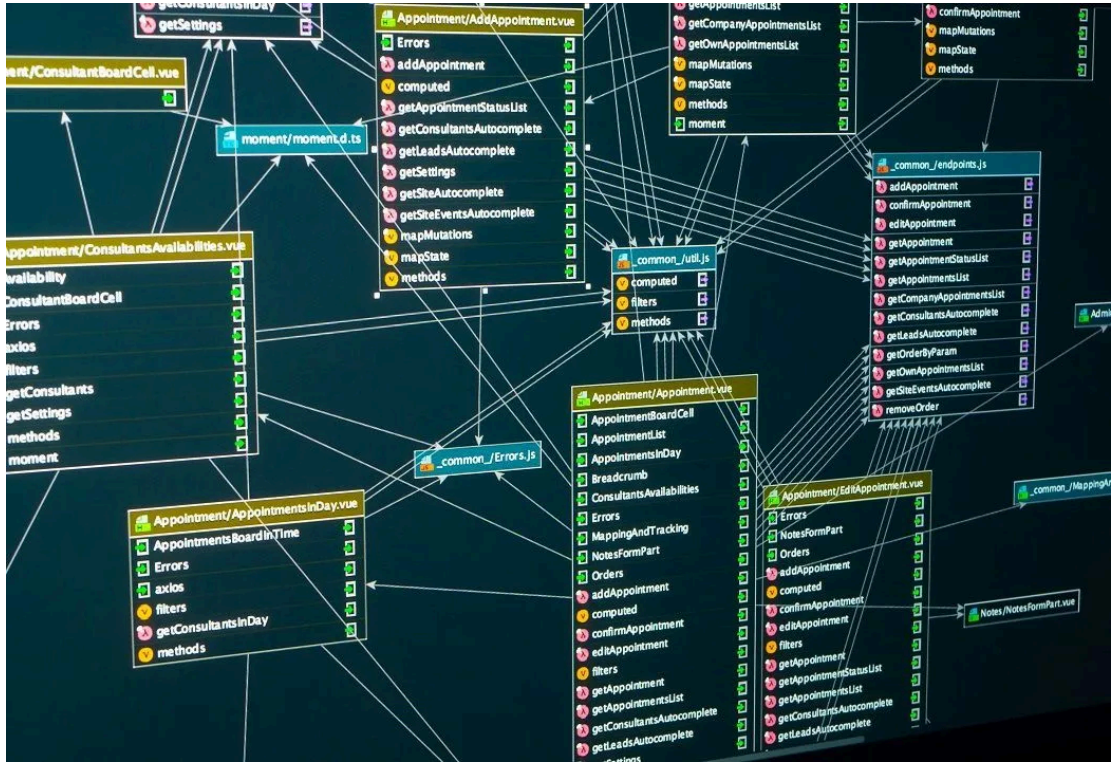
# Tables (CSV)

Narration

# Tabular data




| | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| 1 | Check number | Date | Description | Amount | | | | |
| 2 | 100 | 2/23/2015 | Water bill | $45.00 | | | | |
| 3 | 101 | 3/24/2015 | Power bill | $67.00 | | | | |
| 4 | 102 | 4/20/2015 | Internet Bill | $50.00 | | | | |
| 5 | | | | | | | | |
| 6 | | | | | | | | |
| 7 | | | | | | | | |
| 8 | | | Total: | $162.00 | | | | |
| 9 | | | Check Balance: | $523.00 | | | | |
| 10 | | | Available: | $361.00 | | | | |
| 11 | | | | | | | | |
| 12 | | | | | | | | |
| 13 | | | | | | | | |
| 14 | | | | | | | | |
| 15 | | | | | | | | |
| 16 | | | | | | | | |
| 17 | | | | | | | | |
| 18 | | | | | | | | |



Narration

# What is a table?

- A table stores data in rows and columns
  - **Rows** represent data for a single thing
  - **Columns** represent attributes of those things
- Great for structured data such as measurements, logs, survey responses

Narration

# Excel

- Excel shows tables *visually*
- This is a way to communicate the data to humans
- but our sketches need a structured format for machines to read

Narration

# Tabular data formats

- (Optionally) start with the column names (headers)
- and then one line per row of data
- Columns are separated by a delimiter
  - CSV = comma-separated values
  - TSV = tab-separated values

Narration

# CSV example

- In CSV files, values are separated by commas
- `loadTable(filename, "csv", "header")` can load these into p5.js

```
name,age,city
Alice,30,New York
Bob,25,Los Angeles
Charlie,35,Chicago
```

- CSV is very common but can be hard to read

Narration

# TSV example

- In TSV files, values are separated by tabs
- `loadTable(filename, "tsv", "header")` can load these into p5.js

```
name    age city
Alice   30  New York
Bob 25  Los Angeles
Charlie 35  Chicago
```

- Other delimiters exist too, but CSV and TSV are most common

Narration

# CSV in p5.js

- Load CSV files with `loadTable()`
- As we are working with files, we need to do this in `preload()`
- Remember to put your CSV files in your sketch folder
- Use `"header"` when your first row contains column names
- Then we can loop through rows and read values per column

Narration

# Accessing tabular data

- `.getRowCount()` gives us the number of rows
- We can access rows by index: `table.getRow(0)`
  - And we can get column values by name: `row.get("age")`
  - This gives us a way to read structured data into our sketch
- Alternatively, you can get an array of a column with `table.getColumn("age")`
  - And then loop through that array of values

Narration

# CSV example

- Here we load a CSV and access its values
- Tip: use `join(", ")` to print arrays nicely (with commas inbetween)

```
let CSV_URL = "https://raw.githubusercontent.com/Sipondo/creative-programming-2026/refs/heads/main/lecture_8/data/city_temperatures.csv";
let table;

function preload() {
  // Load our csv into a global variable
  table = loadTable(CSV_URL, "csv", "header");
}

function setup() {
  createCanvas(500, 420);
}

function draw() {
  background(245);
  textSize(20);
```

CSV: rows + columns
Rows: 16 | Columns: 5
Columns: city, week, tempC, humidity, wind

Copenhagen week=1 temp=4C
Copenhagen week=2 temp=6C
Copenhagen week=3 temp=8C
Copenhagen week=4 temp=10C
Aarhus week=1 temp=3C
Aarhus week=2 temp=5C
Aarhus week=3 temp=7C
Aarhus week=4 temp=9C

Narration

# Unstructured access

- We can also access values by row and column index: `table.get(0, 1)`

```
let CSV_URL = "https://raw.githubusercontent.com/Sipondo/creative-programming-2026/refs/heads/main/lecture_8/data/city_temperatures.csv";
let table;

function preload() {
  // Load our csv into a global variable
  table = loadTable(CSV_URL, "csv", "header");
}

function setup() {
  createCanvas(500, 420);
}

function draw() {
  background(245);
  textSize(20);

  // Display some stats
```

```
CSV: rows + columns
Rows: 16 | Columns: 5
Columns: city, week, tempC, humidity, wind

Copenhagen
Copenhagen
4
```

Narration

# Filtering rows

- In real projects, we rarely use all rows at once
- We often filter by category, city, date, or threshold
- Filtering is conditional logic inside a loop

```
let CSV_URL = "https://raw.githubusercontent.com/Sipondo/creative-programming-2026/refs/heads/main/lecture_8/data/city_temperatures.csv";
let table;

let cities = ["Copenhagen", "Aarhus", "Odense", "Aalborg"]; // Hardcoded list of cities. We could also grab this from the table directly!
let cityIndex = 0;

function preload() {
  // Load our csv into a global variable
  table = loadTable(CSV_URL, "csv", "header");
}

```

City: Copenhagen (press SPACE to switch)

week=1 temp=4C
week=2 temp=6C
week=3 temp=8C
week=4 temp=10C

Narration

# Mapping table values to visuals

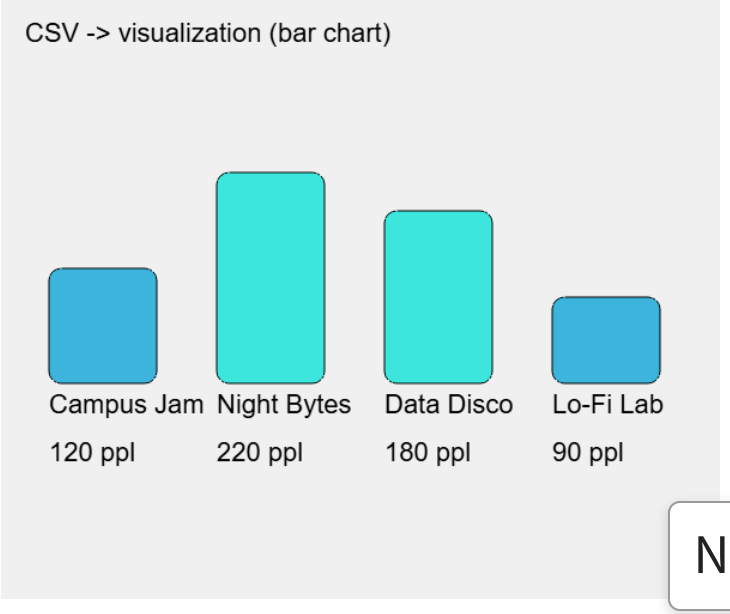
- Data values can drive size, position, and color
- This is where data starts feeling visual and expressive
- Always start simple: one variable -> one visual property

```
let CSV_URL = "https://raw.githubusercontent.com/Sipondo/creative-programming-2026/refs/heads/main/lecture_8/data/music_events.csv";
let events;

function preload() {
  events = loadTable(CSV_URL, "csv", "header");
}

function setup() {
  createCanvas(600, 500);
}

function draw() {
```

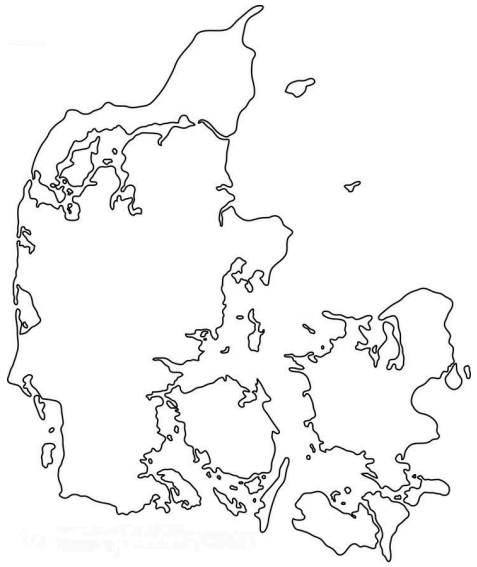



Narration

# CSV challenge

**Class Assignment: Denmark On The Map**

1. Copy the sketch code below, and download the data from SimpleMaps
2. Load the CSV data (`preload()`) and inspect it (e.g. console.log)
3. Draw a simple shape for each city on the map. It's tricky to convert the coordinates to the map!
4. Can you visualise the population as well?



Narration

# JSON

Narration

# Returning to objects

- Last lecture we used objects to structure our code and data
- Objects are a fundamental way to group related data and behavior
- What if we want a lot of objects?
- A seperate data file can help us manage that complexity

Narration

# JSON

- JavaScript Object Notation (JSON) stores objects and arrays in text
- CSV is great for flat tables
- JSON is better for nested structures and mixed data
- JSON is very common in web apps and APIs

Narration

# JSON building blocks

- Objects: key-value pairs (`{ ... }`)
- Arrays: ordered lists (`[ ... ]`)
  - This mimics how we've been writing these two in code!
- JSON combines these to create complex data structures

```json
{
  "name": "Alice",
  "age": 30,
  "city": "New York",
  "hobbies": ["painting", "cycling"],
  "metrics": {
    "score": 85,
    "rank": 5
  }
}
```

Narration

# JSON in action

- We can load JSON files with `loadJSON()` (in `preload()`)
- Remember to put your JSON files in your sketch folder

```
let JSON_URL = "https://raw.githubusercontent.com/Sipondo/creative-programming-2026/refs/heads/main/lecture_8/data/creative_projects.json";
let data;

function preload() {
  // Load our json data tree
  data = loadJSON(JSON_URL);
}

function setup() {
  createCanvas(600, 420);
}

function draw() {
  background(245);
  textSize(20);
```

JSON: objects and arrays
Projects loaded: 4

Mood Garden | team: Axel+Line | tags: visual, interactive
Sound Trails | team: Kirstine+Thomas | tags: audio, motion
Data Bubbles | team: Ties | tags: data, visual
Sprite Stories | team: Nina+Jonas | tags: game, narrative

Narration

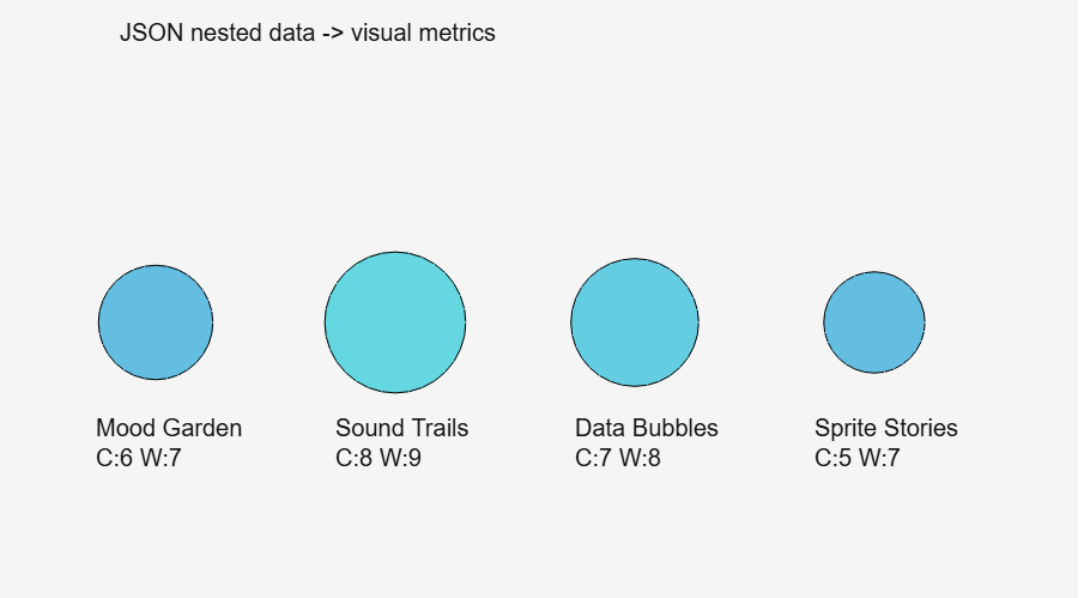
# Nested JSON values

- Nested fields let us store grouped information cleanly
- Access pattern example: `project.metrics.wow`

```
let JSON_URL =
"https://raw.githubusercontent.com/Sipondo/creative-
programming-
2026/refs/heads/main/lecture_8/data/creative_projects.jso
n";
let data;

function preload() {
  // Load the json
  data = loadJSON(JSON_URL);
}

function setup() {
  createCanvas(900, 500);
}
```




Narration

# Filtering JSON arrays

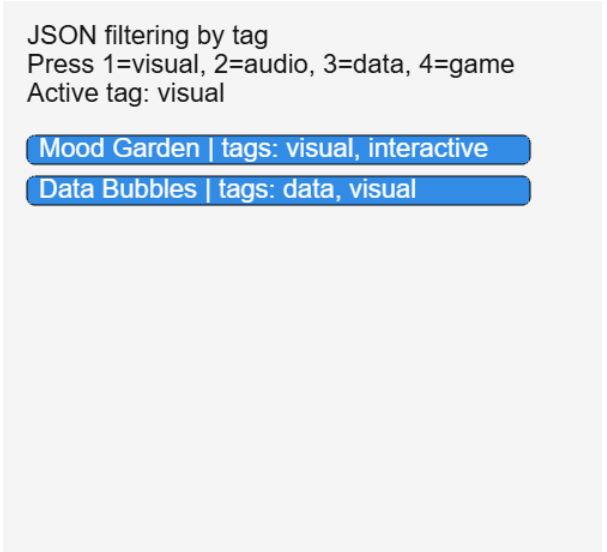
- JSON arrays are arrays, so we can loop through them
- As seen last week, either use `for` or `forEach`

```
let JSON_URL = "https://raw.githubusercontent.com/Sipondo/creative-programming-2026/refs/heads/main/lecture_8/data/creative_projects.json";
let data;
let activeTag = "visual";

function preload() {
  // Load the json
  data = loadJSON(JSON_URL);
}

function setup() {
  createCanvas(500, 460);
}

function draw() {
  background(245);
```




Narration

# APIs

Narration

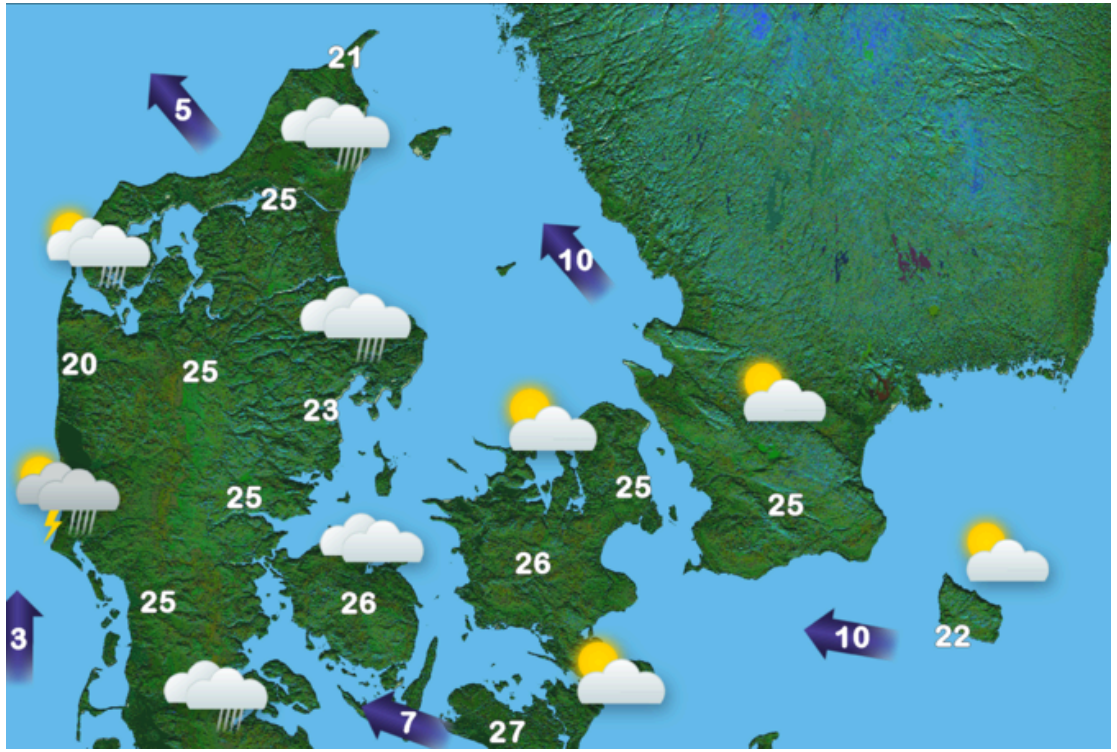
# What is an API?

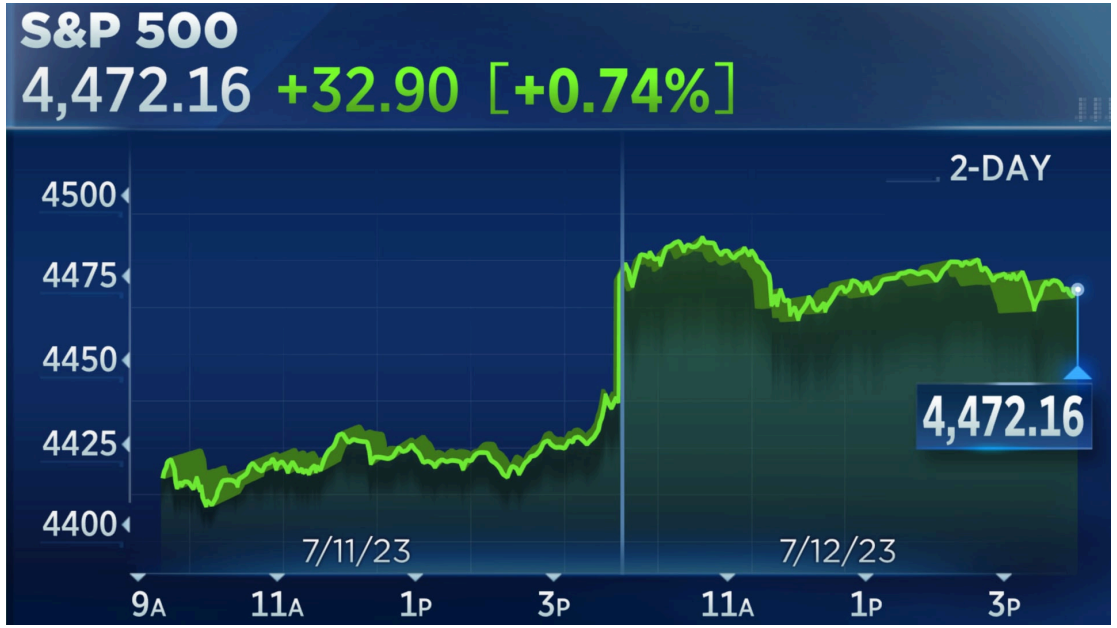
- Application Programming Interface (API)
- A way to request data from another service
- Instead of static local files, we can fetch live data
- The response is often JSON
- We’ve seen this before; we used URLs for images and sounds

Narration

# API examples

- Weather data, news headlines, stock prices, social media posts




Narration

# API request flow

- We can often use APIs the same as json
- `loadJSON()` can fetch from a URL
- Sometimes, APIs are more complex and consist of multiple steps
- For example, you may need to load multiple JSON endpoints

Narration

# First API example

- Here we load some weather data from a public API

```
let API_URL = "https://api.open-meteo.com/v1/forecast?
latitude=52.37&longitude=4.90&current=temperature_2m,wind_speed_10m";
let weather;

function preload() {
  // Load API data before setup/draw start
  weather = loadJSON(API_URL);
}

function setup() {
  createCanvas(560, 240);
  textFont("monospace");
}

function draw() {
  background(245);
  fill(20);
  textSize(22);
```

```
Open-Meteo with loadJSON()

Amsterdam

Temperature: 9.3 C

Wind speed: 15.1 km/h
```

Narration

# API interaction

- We can request new data on click or key press
- This allows dynamic, changing sketches
- User action + data request can become part of the design
- Problem: how can we do this outside of `preload()`?

Narration

# Callbacks

- Want to use APIs outside of `preload()`? Use callbacks!
- Similar syntax to `forEach`
- In this case, run a piece of code when successful, or on error

```
loadJSON(url,
  (data) => {
    // success callback
    console.log("Data loaded:", data);
  },
  (error) => {
    // error callback
    console.error("Error loading data:", error);
  }
);
```

Narration

# Cats!

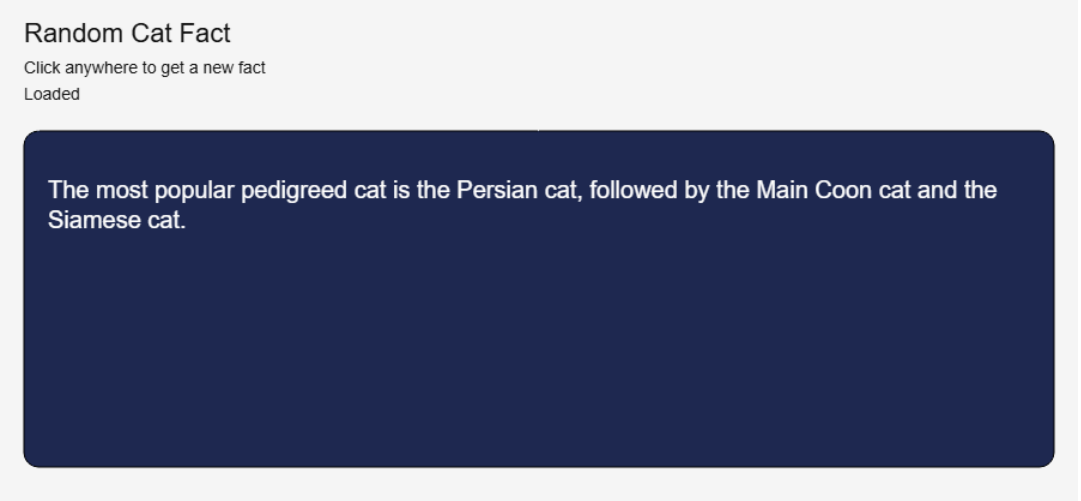
- In this example, we fetch a random cat fact on click
- Tip: use `textWrap(WORD)` to make long text fit better

```
let catFact = "Loading cat fact...";
let statusText = "";

function setup() {
  createCanvas(900, 420);
  // Grab us an initial cat fact
  fetchCatFact();
}

function draw() {
  background(245);

  // Bunch of drawing logic
  fill(20);
  textSize(22);
  text("Random Cat Fact", 20, 36);
```




Narration

# API assignment

### Class Assignment: Weather Alert!

1. Open your Denmark sketch back up (from class assignment 1)
   - Alternatively, use the base map sketch without the CSV data
2. Use the open meteo website to craft a weather API request for one of the cities
3. Display the weather data for that city on the map
4. Can you make the URL dynamic so that you can display the weather for different cities when pressing a button?

```
1  var MAP_URL = "https://rawcdn.githack.com/Sipondo/creative-programming-
   2026/57bf1d0b3f25b4baadde2c986a3f43a736b7afc2/lecture_8/media/denmark.jpg";
2  let denmarkMap;
3
4  function preload() {
5    // Load the map image before setup/draw starts
6    denmarkMap = loadImage(MAP_URL);
```

Narration

# Excellent APIs: Art

- Both the Art Institute of Chicago and Statens Museum for Kunst have well-documented APIs with lots of data and images to explore

```
let artData;
let artImage;
let artIndex = 0;

function preload() {
  // Pick a keyword for the museum API search
  let searchTerm = "flowers"; // keyword that is used in the API call

  // Load JSON before setup/draw runs
  artData = loadJSON(
    "https://api.smk.dk/api/v1/art/search/?keys=" +
      searchTerm +
      "&offset=0&rows=1000&lang=en"
  );
}

```

SMK Art API (flowers)

Narration

# Excellent APIs: Dogs

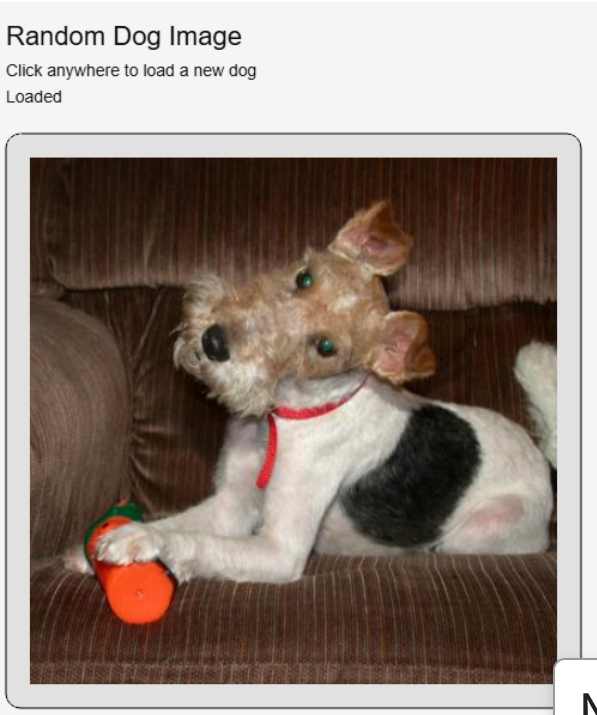
- The Dog API is a fun and simple API that provides random dog images and breed information.

```
let dogImage;
let statusText = "";

function setup() {
  createCanvas(520, 620);
  // Grab us an initial dog image
  fetchDogImage();
}

function draw() {
  background(245);

  // Bunch of drawing logic
  fill(20);
  textSize(22);
  text("Random Dog Image", 20, 36);
```




Narration

# Excellent APIs: Questions

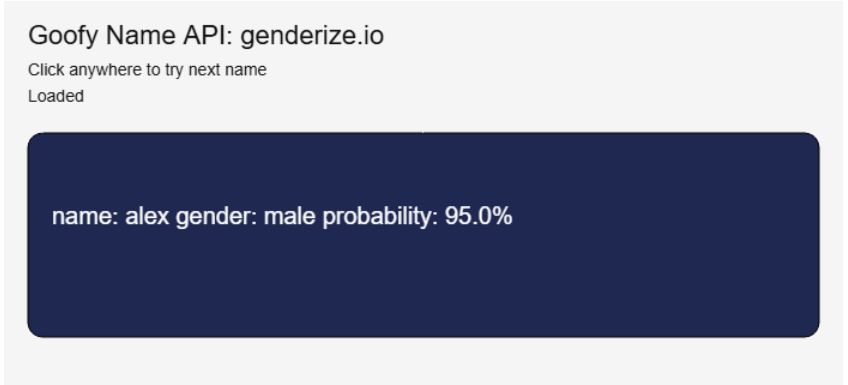
- Genderize, Agify, and Nationalize predict on names
- These APIs require **input**!

```
let names = ["alex", "sam", "maria", "john", "sasha"];
let nameIndex = 0;
let resultText = "Click to predict a name";
let statusText = "";

function setup() {
  createCanvas(700, 320);
  // Grab us an initial prediction
  fetchPrediction();
}

function draw() {
  background(245);

  // Bunch of drawing logic
  fill(20);
```




Narration

# Summary

- CSV: best for flat table-like datasets
- JSON: best for nested and flexible structures
- APIs: best for live and external data

Narration